

Mitali Bhatia

D20C 04

Blockchain

Experiment No 05

Aim : Hands on Solidity Programming Assignments for creating Smart Contracts.

Theory :

Solidity

Solidity is a programming language used to write **Smart Contracts** on the Ethereum blockchain. It supports different data types, variables, functions, control flow statements, arrays, mappings, structures, enums, and transactions involving Ether.

1. Primitive Data Types and Variables

Solidity provides basic data types such as:

- **uint** – stores unsigned integer values.
- **int** – stores positive and negative integer values.
- **bool** – stores **true** or **false**.
- **address** – stores an Ethereum address.
- **string** – stores text.

Variables can be declared as **state variables** or **local variables**.

2. Functions

Functions are used to perform operations in a smart contract.

- **public** – can be accessed from outside the contract.
- **view** – reads data but does not modify the blockchain state.
- **pure** – does not read or modify blockchain state.
- Functions can also take **inputs** and return **outputs**.

3. Visibility, Modifiers and Constructors

- **Visibility** decides who can access a function or variable.
- **Modifiers** are used to add conditions or rules to functions.
- A **constructor** runs automatically when the contract is deployed and is commonly used to initialize values.

4. Control Flow

Solidity supports control flow statements such as:

- `if-else`
- `for` loop
- `while` loop
- `continue`
- `break`
- ternary operator

These are used to control how the smart contract executes.

5. Data Structures

Solidity supports different data structures:

- **Arrays** – store multiple values.
- **Mappings** – store values using key-value pairs.
- **Structs** – group related data together.
- **Enums** – define a fixed set of possible values.

6. Data Locations

For reference types such as arrays and structs, Solidity provides different data locations:

- **storage** – data is stored permanently on the blockchain.
- **memory** – temporary data used during function execution.
- **calldata** – read-only data passed to external functions.

7. Ether, Wei and Gas

Solidity supports transactions involving **Ether**.

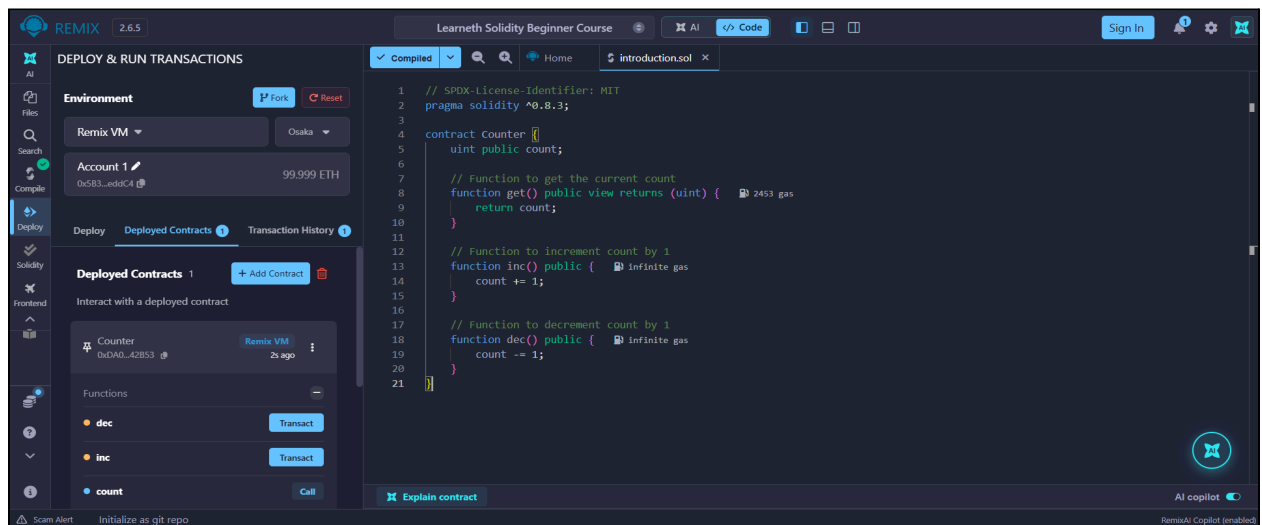
- **Wei** is the smallest unit of Ether.
- **1 Ether = 10^{18} Wei**
- **1 Gwei = 10^9 Wei**
- **Gas** is the computational cost required to execute transactions and smart contract operations.

Ether can be received and sent using payable functions and suitable Ether-transfer methods.

OUTPUT :

Tutorial 1:

Counter Contract Deployed



The screenshot displays the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar shows the environment set to 'Remix VM' and 'Osaka'. A transaction history table lists a 'Counter' contract deployment. The 'Deployed Contracts' section shows the 'Counter' contract with its address '0x0A0...42853'. Below this, the 'Functions' section lists 'dec', 'inc', and 'count', each with a 'Transact' button. The main editor shows the Solidity code for the 'Counter' contract, which includes a 'count' variable, a 'get()' function, and 'inc()' and 'dec()' functions. The status bar at the bottom indicates 'Scam Alert' and 'Initialize as git repo'.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Counter {
5     uint public count;
6
7     // Function to get the current count
8     function get() public view returns (uint) {
9         return count;
10    }
11
12    // Function to increment count by 1
13    function inc() public {
14        count += 1;
15    }
16
17    // Function to decrement count by 1
18    function dec() public {
19        count -= 1;
20    }
21 }
```

Counter Incremented Successfully

The screenshot displays the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar shows the 'Environment' with 'Remix VM' and 'Osaka' selected. Under 'Deployed Contracts', the 'count' contract is listed with a 'Call' button. The 'Transaction History' pane shows a transaction with the value '0x1256: 4'. The main editor displays the Solidity code for a counter contract. The 'Explain contract' pane at the bottom shows a transaction log with the following details:

- Transaction 0: [vm] from: 0x5B3...eddC4 to: Counter.inc() 0x0A0...42B53 value: 0 wei data: 0x371...303c0 logs: 0 hash: 0x73b...9679e
- CALL [call] from: 0x5B380a6a701c568545dCfc803Fc8875F56beddC4 to: Counter.get() data: 0x6d4...ce63c

The status bar at the bottom indicates 'RemixAI Copilot (enabled)'.

Counter Decremented Successfully

The screenshot displays the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' sidebar shows the 'Environment' with 'Remix VM' and 'Osaka' selected. Under 'Deployed Contracts', the 'count' contract is listed with a 'Call' button. The 'Transaction History' pane shows a transaction with the value '0x1256: 2'. The main editor displays the Solidity code for a counter contract. The 'Explain contract' pane at the bottom shows a transaction log with the following details:

- Transaction 0: [vm] from: 0x5B3...eddC4 to: Counter.dec() 0x0A0...42B53 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0x2c3...bc3e2
- CALL [call] from: 0x5B380a6a701c568545dCfc803Fc8875F56beddC4 to: Counter.get() data: 0x6d4...ce63c

The status bar at the bottom indicates 'RemixAI Copilot (enabled)'.

Tutorial 2 :

Remix is moving to app.remix.live. Your Workspaces are stored the browser and won't be accessible from the new URL unless you follow the move instructions. [Move my Workspaces](#)

You can use 'Generate documentation' in the right-click menu to get AI-generated documentation.

REMITX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

2. Basic Syntax 2 / 19

We also define the **visibility** of the variable, which specifies from where you can access it. In this case, it's a **public** variable that you can access from inside and outside the contract.

Don't worry if you didn't understand some concepts like **visibility**, **data types**, or **state variables**. We will look into them in the following sections.

To help you understand the code, we will link in all following sections to video tutorials from the [creator](#) of the Solidity by Example contracts.

[Watch a video tutorial on Basic Syntax.](#)

★ **Assignment**

1. Delete the HelloWorld contract and its content.
2. Create a new contract named "MyContract".
3. The contract should have a public state variable called "name" of the type string.
4. Assign the value "Alice" to your new variable.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2
3 pragma solidity ^0.8.3;
4
5 contract MyContract {
6
7     string public name = "Alice";
8
9 }
```

0 Listen on all transactions Filter with transaction hash or address

[we] from: 0x583...addC4 to: Counter_dec() 0xDAB...42B53 value: 0 wei data: 0xb3b...cfa82 logs: 0 hash: 0x2c3...bc3e2

CALL [call] from: 0xc5838da6a701c568545dcfc803fc8875f56bed9C4 to: Counter_get() data: 0x6d4...ce63c

RemixAI Copilot (enabled)

Tutorial 3 :

Remix is moving to app.remix.live. Your Workspaces are stored the browser and won't be accessible from the new URL unless you follow the move instructions. [Move my Workspaces](#)

You can use 'Generate documentation' in the right-click menu to get AI-generated documentation.

REMITX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

3. Primitive Data Types 3 / 19

For these data types have default values, as shown in the example (line 6-9):

You can learn more about these data types as well as [Fixed Point Numbers](#), [Byte Arrays](#), [Strings](#), and more in the [Solidity documentation](#).

Later in the course, we will look at data structures like [Mappings](#), [Arrays](#), [Enums](#), and [Structs](#).

[Watch a video tutorial on Primitive Data Types.](#)

★ **Assignment**

1. Create a new variable **newAddr** that is a **public** **address** and give it a value that is not the same as the available variable **addr**.
2. Create a **public** variable called **neg** that is a negative number, decide upon the type.
3. Create a new variable, **new0** that has the smallest **uint** size type and the smallest **uint** value and is **public**.

Tip: Look at the other address in the contract or search the internet for an Ethereum address.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2
3 pragma solidity ^0.8.3;
4
5 contract Primitives {
6     bool public boo = true;
7
8     uint8 public u8 = 1;
9     uint public u256 = 456;
10    uint public u = 123;
11
12    int8 public i8 = -1;
13    int public i256 = 456;
14    int public i = -123;
15
16    address public addr = 0xCA35b7d915458F540aDe6068dfe2F44E8fa733c;
17    address public newAddr = 0x0000000000000000000000000000000000000000000000000000000000000001;
18
19    int8 public neg = -1;
20
21    uint8 public new0 = 0;
22
23    bool public defaultBoo;
24    uint public defaultUint;
25    int public defaultInt;
```

0 Listen on all transactions Filter with transaction hash or address

RemixAI Copilot (enabled)

Tutorial 4 :

Remix is moving to app.remix.live. Your Workspaces are stored the browser and won't be accessible from the new URL unless you follow the move instructions. [Move my Workspaces](#)

You can use 'Generate documentation' in the right-click menu to get AI-generated documentation.

LEARNETH

Tutorials list

4. Variables 4 / 19

retrieve information about the blockchain, particular addresses, contracts, and transactions.

In this example, we use `block.timestamp` (line 14) to get a Unix timestamp of when the current block was generated and `msg.sender` (line 15) to get the caller of the contract function's address.

A list of all Global Variables is available in the Solidity documentation.

Watch video tutorials on [State Variables](#), [Local Variables](#), and [Global Variables](#).

★ Assignment

1. Create a new public state variable called `blockNumber`.
2. Inside the function `doSomething()`, assign the value of the current block number to the state variable `blockNumber`.

Tip: Look into the global variables section of the Solidity documentation to find out how to read the current block number.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Variables {
5     // State variables are stored on the blockchain.
6     string public text = "Hello";
7     uint public num = 123;
8
9     // New state variable
10    uint public blockNumber;
11
12    function doSomething() public { 22334 gas
13        // local variable
14        uint i = 456;
15
16        // Global variables
17        uint timestamp = block.timestamp;
18        address sender = msg.sender;
19
20        // Assign current block number
21        blockNumber = block.number;
22    }
23 }
```

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo

RemixAI Copilot (enabled)

Tutorial 5:

LEARNETH

Tutorials list

5.1 Functions - Reading and Writing to a State Variable 5 / 19

If the function takes inputs like our `set` function (line 9), you must specify the parameter types and names. A common convention is to use an underscore as a prefix for the parameter name to distinguish them from state variables.

You can then set the visibility of a function and declare them `view` or `pure` as we do for the `get` function if they don't modify the state. Our `get` function also returns values, so we have to specify the return types. In this case, it's a `uint` since the state variable `num` that the function returns is a `uint`.

We will explore the particularities of Solidity functions in more detail in the following sections.

Watch a video tutorial on [Functions](#).

★ Assignment

1. Create a public state variable called `b` that is of type `bool` and initialize it to `true`.
2. Create a public function called `get_b` that returns the value of `b`.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract SimpleStorage {
5     // State variable to store a number
6     uint public num;
7
8     // Set the value of num
9     function set(uint _num) public { 22536 gas
10         num = _num;
11     }
12
13     // Read the value of num
14     function get() public view returns (uint) { 2475 gas
15         return num;
16     }
17
18     // New boolean state variable
19     bool public b = true;
20
21     // Return the value of b
22     function get_b() public view returns (bool) { 2539 gas
23         return b;
24     }
25 }
```

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo

RemixAI Copilot (enabled)

Tutorial 6:

LEARNETH 2.6.5 Learneth Solidity Beginner Course

Tutorials list 5.2 Functions - View and Pure 6 / 19

From the [Solidity documentation](#).

You can declare a pure function using the keyword `pure`. In this contract, `add` (line 13) is a pure function. This function takes the parameters `i` and `j`, and returns the sum of them. It neither reads nor modifies the state variable `x`.

In Solidity development, you need to optimise your code for saving computation cost (gas cost). Declaring functions `view` and `pure` can save gas cost and make the code more readable and easier to maintain. Pure functions don't have any side effects and will always return the same result if you pass the same arguments.

Watch a video tutorial on View and Pure Functions.

★ Assignment

Create a function called `addToX2` that takes the parameter `y` and updates the state variable `x` with the sum of the parameter and the state variable `x`.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

viewAndPure.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract ViewAndPure {
5     uint public x = 1;
6
7     // Promise not to modify the state.
8     function addToX(uint y) public view returns (uint) { infinite gas
9         return x + y;
10    }
11
12    // Promise not to modify or read from the state.
13    function add(uint i, uint j) public pure returns (uint) { infinite gas
14        return i + j;
15    }
16
17    // Add y to x and update x
18    function addToX2(uint y) public { infinite gas
19        x = x + y;
20    }
21 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

Tutorial 7:

LEARNETH 2.6.5 Learneth Solidity Beginner Course

Tutorials list 5.3 Functions - Modifiers and Constructors 7 / 19

contract initialization code. The constructor can have parameters and is especially useful when you don't know certain initialization values before the deployment of the contract.

You declare a constructor using the `constructor` keyword. The constructor in this contract (line 11) sets the initial value of the owner variable upon the creation of the contract.

Watch a video tutorial on Function Modifiers.

★ Assignment

1. Create a new function, `increaseX` in the contract. The function should take an input parameter of type `uint` and increase the value of the variable `x` by the value of the input parameter.
2. Make sure that `x` can only be increased.
3. The body of the function `increaseX` should be empty.

Tip: Use modifiers.

[Check Answer](#) [Show answer](#)

Next

Well done! No errors.

modifiersAndConstructors.sol

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract FunctionModifier {
5     address public owner;
6     uint public x = 10;
7     bool public locked;
8
9     constructor() { 471829 gas 425800 gas
10         owner = msg.sender;
11     }
12
13     modifier onlyOwner() {
14         require(msg.sender == owner, "Not owner");
15         _;
16     }
17
18     modifier validAddress(address _addr) {
19         require(_addr != address(0), "Not valid address");
20         _;
21     }
22
23     function changeOwner(address _newOwner) public onlyOwner validAddress(_newOwner) {
24         owner = _newOwner;
25     }
26 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

Tutorial 8 :

LEARNETH 2.6.5 Learneth Solidity Beginner Course

5.4 Functions - Inputs and Outputs 8 / 19

Input and Output restrictions

There are a few restrictions and best practices for the input and output parameters of contract functions.

"[Mappings] cannot be used as parameters or return parameters of contract functions that are publicly visible." From the Solidity documentation.

Arrays can be used as parameters, as shown in the function `arrayInput` (line 71). Arrays can also be used as return parameters as shown in the function `arrayOutput` (line 76).

You have to be cautious with arrays of arbitrary size because of their gas consumption. While a function using very large arrays as inputs might fail when the gas costs are too high, a function using a smaller array might still be able to execute.

Watch a video tutorial on Function Outputs.

Assignment

Create a new function called `returnTwo` that returns the values `2` and `true` without using a return statement.

Check Answer **Show answer**

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Function {
5     function returnMany() infinite gas
6     public
7     pure
8     returns (
9         uint,
10        bool,
11        uint
12    )
13 {
14     return (1, true, 2);
15 }
16
17 function named() infinite gas
18 public
19 pure
20 returns (
21     uint x,
22     bool b,
23     uint y
24 )
25 {
26     return (1, true, 2);
27 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo

Tutorial 9 :

LEARNETH 2.6.5 Learneth Solidity Beginner Course

6. Visibility 9 / 19

external

- Can be called from other contracts or transactions
- State variables can not be `external`

In this example, we have two contracts, the `Base` contract (line 4) and the `Child` contract (line 55) which inherits the functions and state variables from the `Base` contract.

When you uncomment the `testPrivateFunc` (lines 58-60) you get an error because the child contract doesn't have access to the private function `privateFunc` from the `Base` contract.

If you compile and deploy the two contracts, you will not be able to call the functions `privateFunc` and `internalFunc` directly. You will only be able to call them via `testPrivateFunc` and `testInternalFunc`.

Watch a video tutorial on Visibility.

Assignment

Create a new function in the `Child` contract called `testInternalVar` that returns the values of all state variables from the `Base` contract that are possible to return.

Check Answer **Show answer**

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Base {
5     function privateFunc() private pure returns (string memory) { infinite gas
6         return "private function called";
7     }
8
9     function testPrivateFunc() public pure returns (string memory) { infinite gas
10        return privateFunc();
11    }
12
13    function internalFunc() internal pure returns (string memory) { infinite gas
14        return "internal function called";
15    }
16
17    function testInternalFunc() public pure virtual returns (string memory) { infinite gas
18        return internalFunc();
19    }
20
21    function publicFunc() public pure returns (string memory) { infinite gas
22        return "public function called";
23    }
24
25    function externalFunc() external pure returns (string memory) { infinite gas
26        return "external function called";
27 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo

Tutorial 10:

LEARNETH

7.1 Control Flow - If/Else

In this contract, the `foo` function uses the `else` statement (line 10) to return `2` if none of the other conditions are met.

else if

With the `else if` statement we can combine several conditions.

If the first condition (line 6) of the `foo` function is not met, but the condition of the `else if` statement (line 8) becomes true, the function returns `1`.

Watch a video tutorial on the If/Else statement.

Assignment

Create a new function called `evenCheck` in the `IfElse` contract:

- That takes in a `uint` as an argument.
- The function returns `true` if the argument is even, and `false` if the argument is odd.
- Use a ternary operator to return the result of the `evenCheck` function.

Tip: The modulo (%) operator produces the remainder of an integer division.

Check Answer **Show answer**

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract IfElse {
5     function foo(uint x) public pure returns (uint) {
6         if (x < 10) {
7             return 0;
8         } else if (x < 20) {
9             return 1;
10        } else {
11            return 2;
12        }
13    }
14
15    function ternary(uint _x) public pure returns (uint) {
16        return _x < 10 ? 1 : 2;
17    }
18
19    // New function
20    function evenCheck(uint _x) public pure returns (bool) {
21        return _x % 2 == 0 ? true : false;
22    }
23 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo

RemixAI Copilot (enabled)

Tutorial 11:

LEARNETH

7.2 Control Flow - Loops

continue

The `continue` statement is used to skip the remaining code block and start the next iteration of the loop. In this contract, the `continue` statement (line 10) will prevent the second if statement (line 12) from being executed.

break

The `break` statement is used to exit a loop. In this contract, the break statement (line 14) will cause the for loop to be terminated after the sixth iteration.

Watch a video tutorial on Loop statements.

Assignment

- Create a public `uint` state variable called `count` in the `Loop` contract.
- At the end of the for loop, increment the count variable by 1.
- Try to get the count variable to be equal to 9, but make sure you don't edit the `break` statement.

Check Answer **Show answer**

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Loop {
5     uint public count;
6
7     function loop() public {
8         // for loop
9         for (uint i = 0; i < 10; i++) {
10             if (i == 3) {
11                 // skip to next iteration with continue
12                 continue;
13             }
14             if (i == 5) {
15                 // Exit loop with break
16                 break;
17             }
18         }
19
20         // while loop
21         uint j;
22         while (j < 9) {
23             j++;
24             count++;
25         }
26     }
27 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo

RemixAI Copilot (enabled)

Tutorial 12:

REMUX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

8.1 Data Structures - Arrays 12 / 19

We can use the `delete` operator to remove an element with a specific index from an array (line 42). When we remove an element with the `delete` operator all other elements stay the same, which means that the length of the array will stay the same. This will create a gap in our array. If the order of the array is not important, then we can move the last element of the array to the place of the deleted element (line 46), or use a mapping. A mapping might be a better choice if we plan to remove elements in our data structure.

Array length

Using the length member, we can read the number of elements that are stored in an array (line 35).

Watch a video tutorial on Arrays.

★ **Assignment**

1. Initialize a public fixed-sized array called `arr3` with the values 0, 1, 2. Make the size as small as possible.
2. Change the `getArr()` function to return the value of `arr3`.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Array {
5     // Several ways to initialize an array
6     uint[] public arr;
7     uint[] public arr2 = [1, 2, 3];
8
9     // Fixed sized array with minimum possible size
10    uint[3] public arr3 = [0, 1, 2];
11
12    uint[10] public myFixedSizeArr;
13
14    function get(uint i) public view returns (uint) {
15        return arr[i];
16    }
17
18    function getArr() public view returns (uint[3] memory) {
19        return arr3;
20    }
21
22    function push(uint i) public {
23        arr.push(i);
24    }
25
26    function pop() public {
27        arr.pop();
28    }
29 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

Tutorial 13:

REMUX 2.6.5 Learneth Solidity Beginner Course

LEARNETH

Tutorials list

8.2 Data Structures - Mappings 13 / 19

Setting values

We set a new value for a key by providing the mapping's name and key in brackets and assigning it a new value (line 16).

Removing values

We can use the delete operator to delete a value associated with a key, which will set it to the default value of 0. As we have seen in the arrays section.

Watch a video tutorial on Mappings.

★ **Assignment**

1. Create a public mapping `balances` that associates the key type `address` with the value type `uint`.
2. Change the functions `get` and `remove` to work with the mapping balances.
3. Change the function `set` to create a new entry to the balances mapping, where the key is the address of the parameter and the value is the balance associated with the address of the parameter.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Mapping {
5     // Mapping from address to uint
6     mapping(address => uint) public balances;
7
8     function get(address _addr) public view returns (uint) {
9         return balances[_addr];
10    }
11
12    function set(address _addr) public {
13        balances[_addr] = _addr.balance;
14    }
15
16    function remove(address _addr) public {
17        delete balances[_addr];
18    }
19 }
20
21 contract NestedMapping {
22     mapping(address => mapping(uint => bool)) public nested;
23
24     function get(address _addr1, uint _i) public view returns (bool) {
25         return nested[_addr1][_i];
26     }
27 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo RemixAI Copilot (enabled)

Tutorial 14:

LEARNETH 2.6.5 Learneth Solidity Beginner Course

8.3 Data Structures - Structs 14 / 19

Key-value mapping: We provide the name of the struct and the keys and values as a mapping inside curly braces (line 19).

Initialize and update a struct: We initialize an empty struct first and then update its member by assigning it a new value (line 23).

Accessing structs

To access a member of a struct we can use the dot operator (line 33).

Updating structs

To update a struct's member we also use the dot operator and assign it a new value (lines 39 and 45).

Watch a video tutorial on Structs.

★ **Assignment**

Create a function `remove` that takes a `uint` as a parameter and deletes a struct member with the given index in the `todos` mapping.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Todos {
5     struct Todo {
6         string text;
7         bool completed;
8     }
9
10    // An array of 'Todo' structs
11    Todo[] public todos;
12
13    function create(string memory _text) public {
14        // 3 ways to initialize a struct
15        todos.push(Todo(_text, false));
16
17        todos.push(Todo({text: _text, completed: false}));
18
19        Todo memory todo;
20        todo.text = _text;
21
22        todos.push(todo);
23    }
24
25    function get(uint _index) public view returns (string memory text, bool completed) {
26        Todo storage todo = todos[_index];
27        return (todo.text, todo.completed);
28    }
29 }
```

Explain contract AI copilot (enabled)

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo

Tutorial 15:

LEARNETH 2.6.5 Learneth Solidity Beginner Course

8.4 Data Structures - Enums 15 / 19

Updating an enum value

We can update the enum value of a variable by assigning it the `uint` representing the enum member (line 30). Shipped would be 1 in this example. Another way to update the value is using the dot operator by providing the name of the enum and its member (line 35).

Removing an enum value

We can use the delete operator to delete the enum value of the variable, which means as for arrays and mappings, to set the default value to 0.

Watch a video tutorial on Enums.

★ **Assignment**

1. Define an enum type called `Size` with the members `S`, `M`, and `L`.

2. Initialize the variable `sizes` of the enum type `Size`.

3. Create a getter function `getSize()` that returns the value of the variable `sizes`.

Check Answer Show answer

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Enum {
5     // Enum representing size
6     enum Size {
7         S,
8         M,
9         L
10    }
11
12    // Initialize variable of enum type
13    Size public sizes;
14
15    // Getter function
16    function getSize() public view returns (Size) {
17        return sizes;
18    }
19 }
```

Explain contract AI copilot (enabled)

0 Listen on all transactions Filter with transaction hash or address

Scam Alert Initialize as git repo

Tutorial 16 :

LEARNETH

Tutorials list

9. Data Locations

16 / 19

If we were to create a new struct `myNewStruct2` with the data location `memory` inside the `function f` (line 12) and assign it the value of `myStruct`, changes in `myNewStruct2` would not affect the values stored in the mapping `myStructs` (line 10).

As we said in the beginning, when creating contracts we have to be mindful of gas costs. Therefore, we need to use data locations that require the lowest amount of gas possible.

★ **Assignment**

1. Change the value of the `myStruct` member `foo`, inside the `function f`, to 4.
2. Create a new struct `myNewStruct2` with the data location `memory` inside the `function f` and assign it the value of `myNewStruct1`. Change the value of the `myNewStruct2` member `foo` to 1.
3. Create a new struct `myNewStruct3` with the data location `memory` inside the `function f` and assign it the value of `myStruct`. Change the value of the `myNewStruct3` member `foo` to 3.
4. Let the function `f` return `myStruct`, `myNewStruct2`, and `myNewStruct3`.

Tip: Make sure to create the correct return types for the function `f`.

Check Answer **Show answer**

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract DataLocations {
5     uint[] public arr;
6     mapping(uint => address) map;
7
8     struct MyStruct {
9         uint foo;
10     }
11
12     mapping(uint => MyStruct) myStructs;
13
14     function f() {
15         // Infinite gas
16         returns (
17             MyStruct memory,
18             MyStruct memory,
19             MyStruct memory
20         )
21     }
22
23     // call f with state variables
24     f(arr, map, myStructs[1]);
25
26     // get a struct from a mapping
27     MyStruct storage myStruct = myStructs[1];
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Tutorial 17:

LEARNETH

Tutorials list

10.1 Transactions - Ether and Wei

17 / 19

`wei` is the smallest subunit of `Ether`, named after the cryptographer Wei Dai. `Ether` numbers without a suffix are treated as `wei` (line 7).

One `gwei` (giga-wei) is equal to 1,000,000,000 (10^9) `wei`.

One `ether` is equal to 1,000,000,000,000,000,000 (10^{18}) `wei` (line 11).

Watch a video tutorial on Ether and Wei.

★ **Assignment**

1. Create a `public uint` called `oneGwei` and set it to 1 `gwei`.
2. Create a `public bool` called `isOneGwei` and set it to the result of a comparison operation between 1 `gwei` and 10^9 .

Tip: Look at how this is written for `gwei` and `ether` in the contract.

Check Answer **Show answer**

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract EtherUnits {
5     uint public oneWei = 1 wei;
6     bool public isOneWei = 1 wei == 1;
7
8     uint public oneEther = 1 ether;
9     bool public isOneEther = 1 ether == 1e18;
10
11     uint public oneGwei = 1 gwei;
12     bool public isOneGwei = 1 gwei == 10**9;
13 }
```

Explain contract

AI copilot

0 Listen on all transactions Filter with transaction hash or address

Tutorial 18 :

LEARNETH 2.6.3 Learneth Solidity Beginner Course

Tutorials list 10.2 Transactions - Gas and Gas Price 18 / 19

Gas prices are denoted in gwei.

Gas limit

When sending a transaction, the sender specifies the maximum amount of gas that they are willing to pay for. If they set the limit too low, their transaction can run out of gas before being completed, reverting any changes being made. In this case, the gas was consumed and can't be refunded.

Learn more about gas on ethereum.org.

Watch a video tutorial on Gas and Gas Price.

Assignment

Create a new **public** state variable in the **Gas** contract called **cost** of the type **uint**. Store the value of the gas cost for deploying the contract in the new variable, including the cost for the value you are storing.

Tip: You can check in the Remix terminal the details of a transaction, including the gas cost. You can also use the Remix plugin *Gas Profiler* to check for the gas cost of transactions.

Check Answer **Show answer**

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Gas {
5     uint public i = 0;
6     uint public cost = 170367;
7
8     // Using up all of the gas that you send causes your transaction to fail.
9     // State changes are undone.
10    // Gas spent are not refunded.
11    function forever() public {
12        // Here we run a loop until all of the gas are spent
13        // and the transaction fails
14        while (true) {
15            i += 1;
16        }
17    }
18 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Tutorial 19 :

LEARNETH 2.6.5 Learneth Solidity Beginner Course

Tutorials list 10.3 Transactions - Sending Ether 19 / 19

If you change the parameter type for the functions **sendViaTransfer** and **sendViaSend** (line 33 and 38) from **payable address** to **address**, you won't be able to use **transfer()** (line 35) or **send()** (line 41).

Watch a video tutorial on Sending Ether.

Assignment

Build a charity contract that receives Ether that can be withdrawn by a beneficiary.

1. Create a contract called **Charity**.
2. Add a public state variable called **owner** of the type **address**.
3. Create a donate function that is public and payable without any parameters or function code.
4. Create a withdraw function that is public and sends the total balance of the contract to the **owner** address.

Tip: Test your contract by deploying it from one account and then sending Ether to it from another account. Then execute the withdraw function.

Check Answer **Show answer**

Next

Well done! No errors.

```
1 // SPDX-License-Identifier: MIT
2 pragma solidity ^0.8.3;
3
4 contract Charity {
5     address public owner;
6
7     constructor() {
8         165452 gas 141800 gas
9         owner = msg.sender;
10    }
11
12    function donate() public payable {}
13
14    function withdraw() public {
15        // Infinite gas
16        uint amount = address(this).balance;
17        (bool sent, bytes memory data) = owner.call(value: amount)("");
18        require(sent, "Failed to send Ether");
19    }
20 }
```

Explain contract AI copilot

0 Listen on all transactions Filter with transaction hash or address

Conclusion:

The fundamentals of **Solidity programming** were explored using **Remix IDE**. Various concepts such as data types, variables, functions, control flow, arrays, mappings, structs, enums, data locations, and Ether transactions were implemented through different assignments. The smart contracts were successfully compiled and deployed using **Remix VM**, helping to understand the basic working of Solidity smart contracts.